password database, allowing an attacker to quickly decrypt all the passwords. Secondly, even if the attacker does not get the encryption key, the encrypted passwords will often be the same length as the original passwords or leak other details about their contents, making it easier for the attacker to guess them. You can solve both problems by storing a *hash* of the password instead. To store the password, you feed it into a one-way hash function that outputs a fixed-length random-looking sequence of bytes. To check if the user has entered the same password, we run their input password through the same hash function and check if it matches the stored hash value. In this way, we never have to store the password itself on the server at all.

The problem with this approach is that because users are not very good at picking unique passwords, it is quite easy for an attacker to pre-compute a table of hashes of common passwords. Once the database has been compromised, the attacker can then very quickly compare the hashes in the database against their lookup table of common password hashes to see if there are any matches. A clever technique known as *rainbow tables* can be used to compress the size of these lookup tables, making this approach even more effective. If that doesn't work, attackers can try a *brute-force* attack of simply trying common passwords and simple variants (known as a *dictionary attack*) or even trying every single possible password up to a given maximum size. Unfortunately, modern computers are extremely fast at computing hash functions, and can easily compute many millions of hashes per second. To make matters worse, computing password hashes is very easy to do in parallel (using lots of CPUs at once) and can be sped up by using the specialist hardware in the GPU of a graphics card or even dedicated chips. For quite small amounts of money (a few thousand dollars), an attacker can easily build a system that can try *billions* of passwords per second, and such hardware can now even be rented from cloud providers when needed.

To make these attacks harder, secure password hashing algorithms employ several countermeasures:

- Adding a random *salt* value to each password before hashing it ensures that each password hash is unique, and an attacker cannot precompute tables of password hashes.
- By iterating the hash function many times, the attacker must expend a lot more CPU time to calculate each guess. Typically, the hash function is repeated 100,000 times or more. Bcrypt and PBKDF2 are examples of iterated password hashes.
- By making the hash function use lots of memory an attacker is prevented from using GPUs and the cost of an attack becomes much greater. Hash functions that use a lot of memory are known as memory-hard. Scrypt and Argon2 are both memory-hard as well as being iterated hashes.

Argon2 (https://en.wikipedia.org/wiki/Argon2) is the winner of the 2015 Password Hashing Competition and represents the state of the art in secure password storage. In this book we will use the older Scrypt design, which is still extremely secure, as it is more widely available at this moment in time.